# 55-ai-docs-spec

## AI-Assisted Documentation & Spec Generation

**Level:** Advanced
**Prerequisites:** Module 38 — AI Code Review & Refactoring
**Output:** AI-generated docs, tech specs, context files, and rule files

### Overview

Dokumentasi dan spesifikasi teknis sering jadi afterthought dalam siklus pengembangan. Modul ini ngajarin cara pakai AI untuk bikin, maintain, dan update dokumentasi secara otomatis — dari tech spec, API docs, context files buat AI agents, sampai rule files buat IDE.

### Learning Outcomes

Setelah modul ini, peserta bisa:

- Generate tech spec lengkap dari PRD pake AI
- Bikin API docs otomatis dari kode (JSDoc/TSDoc, OpenAPI)
- Buat context files (AGENTS.md, CLAUDE.md, SOUL.md) untuk AI agents
- Configure rule files (.cursorrules, copilot-instructions) untuk project
- Setup auto-update docs saat kode berubah

### Sessions

| # | Session | Description |
|---|---|---|
| 01 | Tech Spec with AI | Generate & maintain technical specifications using AI, ADRs, sequence diagrams |

| # | Session | Description |
|---|---|---|
| 02 | Documentation Generation | README, API docs, changelog, auto-update documentation from code changes |
| 03 | Context Files | AGENTS.md, CLAUDE.md, SOUL.md — context files for AI coding agents |
| 04 | Rules Files | .cursorrules, copilot-instructions, IDE config for AI-assisted coding |

## Key Concepts

- **Tech Spec AI**: AI-assisted generation dari PRD → structured tech spec
- **ADR**: Architecture Decision Records — dokumentasi keputusan arsitektur
- **Auto-Docs**: Dokumentasi yang digenerate otomatis dari kode
- **Context Files**: File khusus buat AI agents biar paham konteks project
- **Rule Files**: Konfigurasi IDE/editor buat ngontrol perilaku AI coding tools

## Tools & Technologies

- AI assistants (Claude, ChatGPT, Copilot)
- Documentation generators: TypeDoc, Swagger-JSDoc, JSDoc
- Mermaid.js untuk diagram
- Conventional Commits + changelog generators
- Cursor, Windsurf, VS Code

## Project Structure

```
55-ai-docs-spec/
├── README.md
├── 01-tech-spec-with-ai.md
├── 02-doc-generation.md
├── 03-context-files.md
├── 04-rules-files.md
└── labs/                          # Lab workspace (peserta)
    └── express-api/           # Contoh project Express untuk latihan
```

Figure 1: Banner

### Related Modules

- **Module 38** — AI Code Review & Refactoring (prerequisite)
- **Module 52** — AI-Powered Pull Request Review
- **Module 53** — Test Generation
- **Module 54** — AI CLI Agents

## Session 01: Tech Spec with AI

**Duration:** 90 minutes
**Objective:** Generate, maintain, and validate technical specifications using AI assistance.

---

### 1. Tech Spec Structure

A solid tech spec covers 8 key sections. AI helps generate drafts and fill details iteratively.

#### Standard Tech Spec Template

```
## Context
Mengapa proyek ini diperlukan? Problem apa yang mau diselesaikan?
Apa hubungannya dengan sistem existing?

## Goals
- Measurable outcomes
- What success looks like
- Technical and business goals

## Non-Goals
- What we're NOT doing (explicitly)
- Scope boundaries
- Future considerations

## Architecture
- High-level diagram (Mermaid)
- Component overview
- Data flow
- Key design decisions

## API Design
- Endpoints
- Request/response schemas
```

```
- Error handling
- Authentication/authorization

## Data Model
- Entities and relationships
- Schema definitions
- Migration strategy

## Migration Plan
- Steps from current to target state
- Rollback strategy
- Data integrity checks

## Risks & Mitigations
- Known unknowns
- Failure scenarios
- Contingency plans
```

**AI-Assisted Generation Flow**

```
PRD Document
    │
    ▼
AI Generate → Tech Spec Draft (semua section)
    │
    ▼
Human Review → Edit/Refine per section
    │
    ▼
AI Iterate → Fill details, expand, add examples
    │
    ▼
AI Review → Check completeness, consistency, feasibility
    │
    ▼
Final Tech Spec
```

**Prompt: Generate Tech Spec from PRD**

```
Kamu adalah principal engineer. Dari PRD berikut, generate tech spec
lengkap dengan 8 section (Context, Goals, Non-Goals, Architecture,
API Design, Data Model, Migration Plan, Risks & Mitigations).

PRD:
[tempel PRD]
```

```
Format:
- Tiap section minimal 3 poin
- Architecture section include Mermaid diagram
- API section include example request/response
- Data model section include table definitions
```

---

## 2. Architecture Decision Records (ADR)

ADR mencatat keputusan arsitektur penting beserta konteks dan alternatifnya.

### ADR Template

```
# ADR-[N]: [Title]

## Status
Proposed | Accepted | Deprecated | Superseded

## Context
Problem yang dihadapi, constraints, faktor yang mempengaruhi.

## Decision
Keputusan yang diambil dan rationale detail.

## Options Considered
| Option | Pros | Cons |
|--------|------|------|
| A      | ...  | ...  |
| B      | ...  | ...  |

## Consequences
Apa konsekuensi dari keputusan ini? Trade-offs?

## References
Link ke docs, RFC, PR terkait.
```

### AI Generate ADR Prompt

```
Dari keputusan arsitektur berikut, generate ADR dengan format
lengkap (Context, Decision, Options Considered, Consequences):

Keputusan: [deskripsi keputusan]
```

```
Options yang dipertimbangkan:
1. [option A] — [pros/cons]
2. [option B] — [pros/cons]
3. [option C] — [pros/cons]

Bikin table comparison untuk options. Include rationale
kenapa option terpilih.
```

---

## 3. Sequence Diagram with Mermaid + AI

AI bisa generate Mermaid sequence diagram dari deskripsi teks.

#### Prompt: Text → Mermaid Diagram

```
Dari deskripsi flow berikut, generate Mermaid sequence diagram:

User login → Frontend kirim POST /auth/login → Backend validasi
→ Database check user → Generate JWT → Return token ke frontend
→ Frontend simpan di localStorage

Detail:
- Participant: User, Frontend, Backend, Database
- Include alt/else untuk error case
- Label tiap step dengan nomor
```

#### Example Output

```
sequenceDiagram
    participant U as User
    participant FE as Frontend
    participant BE as Backend
    participant DB as Database

    U->>FE: 1. Submit login form
    FE->>BE: 2. POST /auth/login {email, password}
    BE->>DB: 3. SELECT user WHERE email=?
    DB-->>BE: 4. User data (or null)
    alt User found & password valid
        BE->>BE: 5. Generate JWT
        BE-->>FE: 6. 200 {token, user}
        FE->>FE: 7. Store token in localStorage
        FE-->>U: 8. Redirect to dashboard
```

```
    else User not found
        BE-->>FE: 9. 401 Invalid credentials
        FE-->>U: 10. Show error message
    end
```

**Mermaid Types for Tech Spec**

| Type | Use Case |
|---|---|
| `sequenceDiagram` | API interaction flow |
| `flowchart TD` | System architecture |
| `classDiagram` | Data model / Entities |
| `stateDiagram-v2` | State machine / Workflow |
| `C4Context` | C4 model context diagram |
| `erDiagram` | Entity-Relationship |
| `gitGraph` | Git workflow / branching |

---

## 4. Lint Spec with AI

AI review tech spec for completeness, consistency, and feasibility.

**AI Spec Review Checklist**

| Check | Description |
|---|---|
| Completeness | Semua 8 section ada? Ada yang missing? |
| Consistency | Istilah konsisten? API di spec match data model? |
| Feasibility | Estimasi realistic? Dependencies clear? |
| Clarity | Bagian yang ambiguous? Perlu detail tambahan? |
| Risk Coverage | Risks teridentifikasi? Ada mitigasi? |

**Prompt: AI Review Tech Spec**

```
Review tech spec berikut sebagai senior engineer:

[tempel tech spec]

Check:
1. Completeness: Ada section yang kurang?
2. Consistency: Ada kontradiksi internal?
3. Feasibility: Ada yang technically impossible?
```

```
4. Clarity: Bagian mana yang perlu detail tambahan?
5. Risk: Ada risk yang missed?

Output format:
- [Issue Type] Section: Deskripsi
- [Suggestion] Perbaikan yang disarankan
```

---

## 5. Latihan: Generate Tech Spec from PRD

### Scenario

Kamu dikasih PRD untuk fitur "User Notification Preferences" — user bisa pilih channel notifikasi (email, push, SMS) dan atur preferensi per kategori.

### Steps

1. **Generate Draft**: Tempel PRD di AI prompt → dapat draft tech spec
2. **Review & Edit**: Baca draft, perbaiki bagian yang kurang tepat
3. **Add ADR**: Buat ADR untuk 1 keputusan arsitektur (misal: pilih message queue atau langsung call)
4. **Add Diagram**: Generate Mermaid sequence diagram untuk flow user update preferences
5. **AI Review**: Minta AI review tech spec, iterasi perbaikan

### Deliverable

File `lab/notification-spec.md` yang berisi:

- Tech spec lengkap 8 sections
- Minimal 1 ADR
- Minimal 1 Mermaid diagram (sequence/flowchart)
- Hasil review AI dan perbaikan yang dilakukan

### Template Starter

```
# Tech Spec: User Notification Preferences

## Context
[Generated by AI]

## Goals
[Generated by AI]
```

```
## Non-Goals
[Generated by AI]

## Architecture
```mermaid
flowchart TD
    A[User] --> B[Frontend]
    B --> C[API Gateway]
    C --> D[Notification Service]
    D --> E[Message Queue]
    E --> F[Email Provider]
    E --> G[Push Provider]
    E --> H[SMS Provider]


---

## Key Takeaways

- Tech spec punya 8 section standard yang bisa digenerate AI dari PRD
- ADR dokumentasi keputusan arsitektur dengan context + options + rationale
- Mermaid diagram bisa digenerate AI dari deskripsi teks
- AI bisa review spec untuk completeness, consistency, feasibility
- Iterasi AI → Human → AI hasilkan spec yang lebih baik

# Session 02: Documentation Generation

**Duration:** 90 minutes
**Objective:** Generate README, API docs, changelog, and auto-
update documentation from code changes.

---

## 1. README Generation

README adalah first impression project. AI bisa generate draf awal dari struktur pro

### README Structure

| Section | Description |
|---------|-------------|
| Title + Badge | Nama project, CI status, version, license |
| Description | 1-2 paragraph tentang project |
| Table of Contents | Navigasi untuk README panjang |
| Setup | Prerequisites, install steps, konfigurasi |
```
```

```
| Usage | Contoh penggunaan, CLI commands, screenshots |
| API | Endpoints, request/response (bisa link ke API docs) |
| Architecture | High-level architecture diagram |
| Contributing | Cara kontribusi, code style, PR process |
| License | License information |

### Prompt: Generate README
```

Dari struktur project berikut, generate README.md yang profesional:

Project: [nama project] — [deskripsi singkat] Stack: [tech stack] Structure: [tree output]

Include: - Setup instructions dengan npm/pnpm/yarn - Usage examples - API documentation section (endpoint summary) - Architecture overview - Contributing guidelines - License (MIT)

Gunakan badge untuk: version, license, CI status.

```
## 2. API Documentation

### JSDoc/TSDoc

JSDoc/TSDoc adalah standard untuk dokumentasi kode JavaScript/TypeScript.

```typescript
/**
 * Creates a new user in the system
 *
 * @param email - User email address (must be unique)
 * @param password - User password (min 8 characters)
 * @param role - User role (default: 'user')
 * @returns The created user object
 * @throws {ValidationError} When email format is invalid
 * @throws {DuplicateError} When email already exists
 *
 * @example
 * ```ts
 * const user = await createUser('john@example.com', 'secure123', 'admin');
 * // => { id: 1, email: 'john@example.com', role: 'admin' }
 * ```
 */
async function createUser(
  email: string,
  password: string,
  role?: UserRole
): Promise<User>
```
```

**TSDoc Tags Reference**

| Tag | Usage |
|---|---|
| `@param` | Document parameter (name + description + type) |
| `@returns` | Document return value |
| `@throws` | Document error conditions |
| `@example` | Show usage example |
| `@deprecated` | Mark as deprecated with reason |
| `@see` | Link to related code |
| `@typeParam` | Document generic type parameter |

**OpenAPI / Swagger**

OpenAPI spec bisa digenerate dari kode atau ditulis manual, lalu AI bantu generate.

```yaml
# Petikan generated oleh swagger-jsdoc
paths:
  /users:
    post:
      summary: Create a new user
      tags: [Users]
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/CreateUserRequest'
      responses:
        '201':
          description: User created successfully
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/User'
        '400':
          description: Validation error
        '409':
          description: Duplicate email
```

**Tools**

| Tool | Description | Command |
|---|---|---|
| TypeDoc | Generate HTML docs from TypeScript | `npx typedoc src/index.ts` |
| swagger-jsdoc | Generate OpenAPI spec from JSDoc | `npx swagger-jsdoc -d swagger.js` |
| jsdoc-to-markdown | JSDoc → Markdown | `npx jsdoc2md src/*.js > API.md` |

#### AI Enhance Docs

```
Dari kode berikut, generate JSDoc comments untuk tiap fungsi:

[kode tanpa JSDoc]

Format:
- @param untuk setiap parameter
- @returns untuk return value
- @throws untuk error cases
- @example untuk usage example
- Deskripsi dalam Bahasa Indonesia
```

---

## 3. Changelog Generation

Conventional Commits → automated changelog.

#### Conventional Commits Format

```
<type>(<scope>): <description>

[body]
[footer]
```

| Type | Release | Changelog Section |
|---|---|---|
| `feat` | MINOR | Features |
| `fix` | PATCH | Bug Fixes |
| `BREAKING CHANGE` foot | MAJOR | Breaking Changes |
| `docs` | - | Documentation |
| `refactor` | - | Code Refactoring |
| `perf` | - | Performance |
| `test` | - | Tests |
| `chore` | - | Chores |

### Tools

```
# standard-version
npx standard-version

# changelogen (pilihan buat monorepo)
npx changelogen --from=v1.0.0 --to=v2.0.0

# git-cliff (Rust, fast)
npx git-cliff -o CHANGELOG.md
```

### AI Generate Changelog

```
Dari git log berikut, generate CHANGELOG.md yang rapi:

[output git log --oneline]

Format:
- Group by type (Features, Bug Fixes, Documentation, etc.)
- Tiap entry ada PR number kalo ada (#123)
- Human-readable descriptions (bukan commit message mentah)
- Versi berdasarkan conventional commits
```

---

## 4. AI Auto-Update Documentation

Siklus: Code change → AI detect → Suggest update → Generate PR.

### Workflow

```
1. Developer commit code change
2. CI/CD atau pre-commit hook trigger AI check
3. AI bandingkan old vs new API surface
4. AI generate suggested doc updates
5. Format sebagai PR atau patch
6. Developer review & merge
```

### Implementation

```yaml
# .github/workflows/docs-auto-update.yml
name: Auto-Update Docs
on:
  pull_request:
    types: [opened, synchronize]
```

```yaml
jobs:
  docs-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Check API changes
        run: |
          # Diff API surface
          git diff origin/main -- src/ > api-diff.txt

      - name: AI Generate Doc Updates
        run: |
          # Kirim diff ke AI → dapatkan suggested doc changes
          ai-cli suggest-docs --diff api-diff.txt

      - name: Create Doc PR
        run: |
          # Auto-create PR dengan doc updates
          gh pr create --title "docs: auto-update from API changes" \
                       --body "AI-generated doc updates" \
                       --base main
```

**Prompt: Detect & Suggest Doc Update**

```
Dari diff code berikut, identifikasi API surface changes
dan generate suggested doc updates:

Diff:
[diff output]

Check:
1. Ada endpoint baru? → Generate OpenAPI spec
2. Ada parameter baru? → Update JSDoc/TSDoc
3. Ada field baru di response? → Update response example
4. Ada behavior change? → Update README usage section

Output:
- List perubahan yang terdeteksi
- Code block for each doc update
- PR description yang ready-to-use
```

---

## 5. Latihan: Generate README + API Docs dari Express Code

### Scenario

Kamu punya Express API sederhana untuk task management. Generate dokumentasi lengkap.

### Starter Code

```
// src/routes/tasks.js
const express = require('express');
const router = express.Router();

router.get('/', async (req, res) => {
  const tasks = await Task.find({ userId: req.user.id });
  res.json(tasks);
});

router.post('/', async (req, res) => {
  const task = await Task.create({
    ...req.body,
    userId: req.user.id
  });
  res.status(201).json(task);
});

router.put('/:id', async (req, res) => {
  const task = await Task.findOneAndUpdate(
    { _id: req.params.id, userId: req.user.id },
    req.body,
    { new: true }
  );
  if (!task) return res.status(404).json({ error: 'Task not found' });
  res.json(task);
});

router.delete('/:id', async (req, res) => {
  const task = await Task.findOneAndDelete({
    _id: req.params.id, userId: req.user.id
  });
  if (!task) return res.status(404).json({ error: 'Task not found' });
  res.status(204).send();
});
```

### Steps

1. **Generate README**: Prompt AI dengan struktur project & hasil tree
2. **Add JSDoc**: Tambah komentar JSDoc ke tiap endpoint pakai AI
3. **Generate OpenAPI**: Setup swagger-jsdoc, generate spec
4. **Generate Changelog**: Buat 5+ conventional commits, lalu generate changelog
5. **AI Auto-Update Demo**: Simulasi code change + AI suggest doc update

### Deliverable

Folder `labs/express-api/docs/` yang berisi:

- `README.md` — Complete README with setup, usage, API
- `API.md` — API documentation (bisa dari swagger atau jsdoc2md)
- `CHANGELOG.md` — Generated changelog
- `suggested-update.md` — Contoh AI-suggested doc update dari code change

---

## Key Takeaways

- README punya struktur baku yang bisa digenerate AI dari project structure
- JSDoc/TSDoc standard untuk inline code documentation
- OpenAPI spec bisa digenerate dari kode atau JSDoc
- Conventional Commits enable automated changelog generation
- AI auto-update docs detect code changes & suggest updates

# Session 03: Context Files for AI Agents

**Duration:** 60 minutes
**Objective:** Create AGENTS.md, CLAUDE.md, and SOUL.md — context files that help AI agents understand project context.

---

## 1. What Are Context Files?

Context files adalah markdown files yang AI coding agents baca untuk paham konteks project. Mereka menjawab: "Apa project ini, gimana strukturnya, apa convention-nya, gimana cara testing?"

### Kenapa Penting?

- **Tanpa context**: AI generate kode yang inconsistent dengan project
- **Dengan context**: AI paham arsitektur, pola, dan constraint project
- **Hasil**: Generate code yang lebih akurat, less hallucination

### Tiga File Utama

| File | Target AI | Fokus |
|---|---|---|
| `AGENTS.md` | All AI agents (general) | Project overview, structure, conventions |
| `CLAUDE.md` | Claude Code (Anthropic) | Coding style, test strategy, preferred patterns |
| `SOUL.md` | Codex (Cursor) | Project soul, constraints, goals |

---

## 2. AGENTS.md — Universal Context

AGENTS.md adalah context file universal yang dibaca oleh berbagai AI coding agents.

### Template

```
# Project Name

## Description
[1-2 paragraph tentang project]

## Repo Structure
```

src/ components/ # React components pages/ # Route pages api/ # API routes lib/ # Utilities & helpers types/ # TypeScript types

```
## Tech Stack
- **Runtime:** Node.js 22
- **Framework:** Next.js 15 (App Router)
- **Language:** TypeScript 5.x
- **Database:** PostgreSQL via Drizzle ORM
```

```
- **Auth:** NextAuth v5
- **UI:** Tailwind CSS v4 + shadcn/ui
- **Testing:** Vitest + Playwright
- **Package Manager:** pnpm

## Architecture
- Monorepo with apps/web, apps/api, packages/*
- Feature-based folder structure dalam apps/web
- Shared types dan utilities di packages/*
- API routes pake Next.js Route Handlers
- Database migration pake Drizzle Kit

## Environment Setup
1. Clone: `git clone git@github.com:org/project.git`
2. Install: `pnpm install`
3. Copy env: `cp .env.example .env.local`
4. Run DB: `docker compose up -d`
5. Migrate: `pnpm db:migrate`
6. Dev: `pnpm dev`

## Conventions
- **Naming:** PascalCase untuk components, camelCase untuk functions
- **Imports:** Group: external → internal → types. Sort alphabetically.
- **Error handling:** throw new AppError(code, message) —
jangan throw Error langsung
- **State management:** Server components preferred. Client only when needed.
- **CSS:** Tailwind utility classes. No CSS modules.
- **Commits:** Conventional Commits (feat:, fix:, chore:, docs:)

## Common Tasks

### Adding a New Page
1. Create file in `src/app/[route]/page.tsx`
2. Add to navigation config di `src/lib/navigation.ts`
3. Write page component (prefer server component)
4. Add E2E test di `e2e/`

### Adding a New API Route
1. Create file in `src/app/api/[route]/route.ts`
2. Define validation schema with Zod
3. Implement handler
4. Add API test di `src/__tests__/api/`

### Database Migration
- `pnpm db:generate` — generate migration from schema changes
- `pnpm db:migrate` — apply pending migrations
```

- `pnpm db:seed` — seed data for development

---

## 3. CLAUDE.md — Claude Code Specific

CLAUDE.md digunakan oleh Claude Code (Anthropic's CLI coding agent). Lebih fokus pada coding style, test strategy, dan patterns.

### Template

```
# CLAUDE.md — Project Guidelines for Claude Code

## Coding Style
- **Formatting:** Prettier default config (tab width: 2, single quotes, no semic
- **TypeScript:** Strict mode. No `any`. Use `unknown` + type narrowing.
- **Async:** Prefer `async/await` over `.then()/.catch()`. No callback patterns.
- **Error handling:** Use `Result<T, E>` pattern (neverthrow or custom). No try/
- **Null checks:** `??` over `||`. `?.` for optional chaining.

## Testing Strategy
- **Unit tests:** Vitest. Test business logic, not implementation details.
- **Integration tests:** Supertest for API routes, msw for external HTTP.
- **E2E tests:** Playwright. Only for critical user flows.
- **Coverage target:** 80%+. Focus on critical paths first.
- **Mocking:** Prefer dependency injection over mocking modules. Use `vi.mock` o
- **Test file location:** Co-located with source: `component.tsx` → `component.t

## Preferred Patterns

### React Components
```tsx
// Prefer server components
async function UserProfile({ userId }: { userId: string }) {
  const user = await getUser(userId);
  return <div>{user.name}</div>;
}

// Client components only when needed
'use client';
function InteractiveButton({ onClick }: { onClick: () => void }) {
  return <button onClick={onClick}>Click</button>;
}
```
```

**API Route Handlers**

```
export async function GET(request: NextRequest) {
  const schema = z.object({ limit: z.coerce.number().default(10) });
  const params = schema.parse(Object.fromEntries(request.nextUrl.searchParams));

  const result = await getUsers(params);
  return NextResponse.json(result);
}
```

**Database Queries**

```
// Use Drizzle queries, not raw SQL
const users = await db.select().from(usersTable).where(eq(usersTable.active, tru
```

## Commands

- `pnpm dev` — Start dev server
- `pnpm test` — Run vitest
- `pnpm test:e2e` — Run Playwright
- `pnpm lint` — ESLint + Prettier check
- `pnpm typecheck` — `tsc --noEmit`
- `pnpm build` — Build for production

## Common Pitfalls

1. Jangan pake `useEffect` untuk data fetching — use server component atau React Query
2. Jangan import dari `@/server/*` di client component
3. Jangan skip error boundaries — wrap new pages with `<ErrorBoundary>`
4. Jangan hardcode environment variables — always use `env.ts` schema

---

## 4. SOUL.md — Codex Specific (Cursor)

SOUL.md digunakan oleh Codex di Cursor editor. Fokus pada "soul" project — konteks, constraints, goals.

### Template

```markdown
# SOUL.md — Project Context for Codex
```

```
## Project Soul
Ini adalah platform manajemen task real-time untuk tim kecil.
Prioritas: kecepatan, UX yang intuitif, dan data consistency.

## Constraints
- **Mobile-first**: All pages must work on 320px+ screens
- **Offline resilience**: Critical operations must queue and retry
- **SSR preferred**: Minimize client JS for SEO and performance
- **GDPR compliance**: User data deletion must cascade to all services
- **Budget**: Free tier PostgreSQL, so queries must be optimized

## Goals
1. **Q1:** Ship MVP with core CRUD + real-time collaboration
2. **Q2:** Add file attachments + search
3. **Q3:** Add integrations (Slack, Discord webhooks)
4. **Q4:** Public API + rate limiting

## Non-Goals
- No native mobile app (PWA is enough)
- No custom auth (NextAuth covers it)
- No real-time sync from 3rd party (polling is fine)

## Architecture Invariants
- Server components are default — client components are explicit opt-
in
- All data access through `src/lib/data-access/*` —
no direct DB calls in route handlers
- UI components in `src/components/ui/*` are pure —
no data fetching
- Business logic in `src/lib/services/*` — testable without HTTP

## References
- Design system: Figma link
- API spec: OpenAPI link
- Database schema: db/schema.ts
```

---

## 5. Perbedaan & Strategi Penggunaan

### Kapan Pake Yang Mana?

| Skenario | File | Alasan |
|---|---|---|
| Claude Code CLI | `CLAUDE.md` | Claude Code otomatis baca ini |
| Cursor AI | `SOUL.md` | Codex di Cursor pake SOUL.md |
| GitHub Copilot Chat | `AGENTS.md` + `.github/copilot-instructions.md` | Copilot baca instructions |
| Multiple AI tools | All three | Tiap file serve different AI |
| Open source project | `AGENTS.md` | Universal, tool-agnostic |

### Overlap & Merge Strategy

Ada overlap besar antara ketiga file:

```yaml
Overlap areas:
  - Tech stack: ada di semua file → referensi aja
  - Repo structure: ada di semua file → referensi aja
  - Coding style: AGENTS.md (umum) vs CLAUDE.md (detail)

Strategy:
  - AGENTS.md: high-level overview buat human & AI
  - CLAUDE.md: detail teknis buat Claude Code
  - SOUL.md: philosophy & constraints buat Codex
```

### Recommended Setup

```
project-root/
├── AGENTS.md              # Universal context (human + AI)
├── CLAUDE.md               # Claude Code specific
├── SOUL.md                 # Codex/Cursor specific
├── .github/
│   └── copilot-instructions.md  # GitHub Copilot
├── .cursorrules            # Cursor rules (next session)
└── .windsurfrules         # Windsurf rules (next session)
```

---

## 6. Template per Stack

### Node/Express

```
# AGENTS.md — Express API

## Tech Stack
- **Runtime:** Node.js 22
- **Framework:** Express.js 4.x
- **Language:** JavaScript (ES2024)
- **Database:** PostgreSQL + Knex.js
- **Auth:** JWT + Passport.js
- **Testing:** Jest + Supertest
- **Validation:** Joi
- **Logging:** Pino

## Conventions
- Route handlers in `src/routes/*.js`
- Controllers in `src/controllers/*.js`
- Middleware in `src/middleware/*.js`
- Error handling: centralized `errorHandler` middleware
- Response format: `{ success: boolean, data?: any, error?: string }`
```

### React/Next.js

```
# AGENTS.md — Next.js Project

## Tech Stack
- **Framework:** Next.js 15 App Router
- **Language:** TypeScript 5.x (strict)
- **UI:** Tailwind CSS v4 + shadcn/ui
- **State:** Server Components + React Query
- **Database:** Prisma ORM + PostgreSQL
- **Auth:** Clerk
- **Testing:** Vitest + Testing Library + Playwright

## Conventions
- `page.tsx` — server component by default
- `layout.tsx` — shared layout
- `loading.tsx` — Suspense boundary
- `error.tsx` — Error boundary
- API routes in `src/app/api/*`
```

**Mastra AI**

```
# AGENTS.md — Mastra AI Project

## Tech Stack
- **Framework:** Mastra AI
- **Runtime:** Node.js 22 + TypeScript
- **Agents:** src/agents/*.ts
- **Tools:** src/tools/*.ts
- **Workflows:** src/workflows/*.ts
- **Memory:** PostgreSQL + pgvector
- **LLM:** OpenAI GPT-4o / Claude 3.5 Sonnet

## Conventions
- Each agent is a class extending `Agent`
- Tools are standalone functions with Zod schemas
- Workflows use `step()` pattern
- RAG pipelines in `src/pipelines/*.ts`
```

**Docker/K8s**

```
# AGENTS.md — Docker + Kubernetes Project

## Tech Stack
- **Container:** Docker + Docker Compose
- **Orchestration:** Kubernetes (k3s local)
- **Infrastructure as Code:** Terraform
- **CI/CD:** GitHub Actions + ArgoCD
- **Monitoring:** Prometheus + Grafana
- **Logging:** Loki + Promtail

## Conventions
- Multi-stage Docker builds
- Docker Compose for local dev with hot-reload
- K8s manifests in `k8s/overlays/` (Kustomize)
- Helm charts for production deployments
- Health checks wajib di semua service
```

---

## 7. Latihan: Bikin AGENTS.md + CLAUDE.md untuk Express Project

### Scenario

Buat context files untuk Express API project task management dari sesi sebelumnya.

### Steps

1. **Bikin AGENTS.md**: Include project desc, repo structure, tech stack, env setup, conventions, common tasks
2. **Bikin CLAUDE.md**: Include coding style, test strategy, preferred patterns, commands, common pitfalls
3. **Bikin SOUL.md**: Include project soul, constraints, goals, architecture invariants
4. **Review**: Minta AI review ketiga file, identifikasi redundansi

### Deliverable

File di `labs/express-api/`:

- `AGENTS.md`
- `CLAUDE.md`
- `SOUL.md`
- `context-review.md` — Analisis overlap & saran merge

---

## Key Takeaways

- **AGENTS.md** — universal context untuk semua AI agents dan human
- **CLAUDE.md** — spesifik Claude Code, detail coding style & patterns
- **SOUL.md** — spesifik Codex/Cursor, filosofi & constraints project
- Tiap file punya target AI berbeda, tapi ada overlap yang bisa dimerge
- Context files drastically improve AI output quality dan consistency
- Template per stack bikin onboarding AI agents lebih cepat

# Session 04: Rules Files for AI-Assisted Coding

**Duration:** 60 minutes
**Objective:** Configure .cursorrules, .windsurfrules, copilot-instructions, and VS Code settings for AI-assisted coding.

---

## 1. What Are Rules Files?

Rules files adalah konfigurasi yang ngontrol behavior AI coding tools di IDE/editor. Mereka define coding style, naming conventions, test patterns, import order, error handling patterns — semua yang AI perlu tau biar generate code yang sesuai dengan project.

### Kenapa Rules Files?

- **Tanpa rules**: AI generate code dengan gaya random
- **Dengan rules**: AI generate code yang match project convention
- **Scalable**: Sekali setup, semua developer di team dapat benefit

### File-File Rules

| File | Tool | Location |
|---|---|---|
| `.cursorrules` | Cursor IDE | Root project |
| `.windsurfrules` | Windsurf IDE | Root project |
| `.github/copilot-instructions.md` | GitHub Copilot | `.github/` |
| `.vscode/settings.json` | VS Code | `.vscode/` |

---

## 2. .cursorrules

Cursor IDE membaca `.cursorrules` untuk guide AI behavior.

### Format

```
You are an expert in [tech stack].

Key instructions:
- [rule 1]
- [rule 2]
```

```
- [rule 3]

Code style:
- [style rule 1]
- [style rule 2]
- [style rule 3]

Testing:
- [test rule 1]
- [test rule 2]

Project-specific:
- [project rule 1]
- [project rule 2]
```

**Example: TypeScript Next.js Project**

```
You are an expert in TypeScript, Next.js 15 (App Router), React 19, Tailwind CSS v4,

Key instructions:
- Write concise, type-safe TypeScript code
- Prefer server components over client components
- Use 'use client' directive only for interactive components
- Always use Zod for input validation
- Use Drizzle ORM for database queries, never raw SQL
- Follow Next.js App Router conventions

Code style:
- PascalCase for components and types
- camelCase for functions, variables, and files
- Single quotes for strings
- No semicolons (Prettier default)
- 2-space indentation
- Sort imports: external → internal → types
- Use named exports, no default exports (except for pages)

Error handling:
- Use Result pattern (neverthrow lib) for business logic
- Use AppError class for known error types
- Let error boundary handle unexpected errors
- Centralized error handling in middleware

Testing:
- Vitest for unit and integration tests
- Playwright for E2E tests
```

```
- Co-locate test files with source files
- Test business logic, not implementation details
- Mock external services with msw

Naming:
- Components: UserProfile.tsx, NotificationsList.tsx
- Pages: page.tsx, layout.tsx, loading.tsx, error.tsx
- API routes: route.ts inside app/api/[route]/
- Utilities: formatDate.ts, validateEmail.ts
- Types: types/user.ts, types/api.ts

Project-specific:
- Database migrations go in db/migrations/
- Environment variables validated in src/lib/env.ts
- API responses format: { success: boolean, data?: T, error?: string }
- All user-facing text must support i18n (next-intl)
- Console.log is forbidden — use logger from src/lib/logger.ts
```

**Example: Express API Project**

```
You are an expert in Node.js 22, Express.js, PostgreSQL, and REST API design.

Key instructions:
- Write clean, modular JavaScript (ES2024)
- Use ES modules (import/export), not CommonJS
- Always validate request data with Joi schemas
- Implement proper error handling middleware
- Use async/await for all asynchronous operations

Code style:
- camelCase for functions and variables
- PascalCase for classes and constructor functions
- Semicolons required
- 2-space indentation
- Arrow functions preferred over function declarations
- Early returns for guard clauses

Project structure:
- routes/ → route definitions only (thin)
- controllers/ → request handling logic
- services/ → business logic
- middleware/ → Express middleware
- models/ → database models (Knex)
- validators/ → Joi schemas
- utils/ → helper functions
```

```
Error handling:
- Centralized errorHandler middleware in app.js
- Custom AppError class with statusCode and code
- All async routes wrapped in asyncHandler
- Never expose stack traces in production

API conventions:
- RESTful endpoints (plural nouns: /users, /tasks)
- Version prefix: /api/v1/
- Standard response format: { success: boolean, data?: any, error?: string }
- Pagination: ?page=1&limit=20 → { data, pagination: { page, limit, total, pages } }
- HTTP status codes: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forb

Testing:
- Jest + Supertest for integration tests
- Test files: *.test.js co-located or in __tests__/
- Each endpoint test covers: success case, validation error, auth error, not found
- Mock database with test containers or in-memory SQLite
```

---

## 3. .windsurfrules

Windsurf (Codeium) menggunakan `.windsurfrules` — mirip cursor rules tapi dengan format sedikit berbeda.

**Format**

```yaml
# .windsurfrules
rules:
  - description: "Use TypeScript strict mode"
    pattern: "*.ts"
    rule: "Always enable strict mode in tsconfig.json. No 'any' type."

  - description: "Component naming"
    pattern: "src/components/**/*.tsx"
    rule: "Use PascalCase for component file names and export names."

  - description: "API route pattern"
    pattern: "src/app/api/**/route.ts"
    rule: "Each route file exports GET, POST, PUT, DELETE handlers. Validate inp

  - description: "Import order"
    pattern: "*.{ts,tsx}"
    rule: "Group: 1) external packages 2) internal modules (@/) 3) types. Sort a
```

```yaml
  - description: "No console.log"
    pattern: "*.{ts,tsx}"
    rule: "Use the project's logger instead of console.log. Console.log fails CI

  - description: "Error boundary"
    pattern: "src/app/**/page.tsx"
    rule: "Wrap new pages with <ErrorBoundary>. Export error.tsx for each route
```

**Windsurf vs Cursor Rules**

| Aspect | .cursorrules | .windsurfrules |
|---|---|---|
| Format | Free text / Markdown | YAML with pattern matching |
| Scoping | Global per file | Per-file-pattern rules |
| Rule granularity | Top-down instructions | Per-glob pattern rules |
| Complexity | Simpler, more narrative | Structured, scoped |

---

## 4. .github/copilot-instructions.md

GitHub Copilot membaca file ini untuk instructions saat generate code atau chat.

**Format**

```
# Copilot Instructions

## Language & Framework
- Language: TypeScript 5.x (strict mode)
- Framework: Next.js 15 (App Router)
- Database: PostgreSQL + Drizzle ORM
- UI: Tailwind CSS v4 + shadcn/ui

## Code Style
- Use TypeScript strict mode with proper type annotations
- Prefer `const` over `let`. Avoid `var`.
- Use template literals over string concatenation
- Destructure objects and arrays when accessing multiple properties
- Use `Array.prototype` methods (.map, .filter, .reduce) over imperative loops

## Naming Conventions
- PascalCase: Components, Types, Interfaces, Classes
- camelCase: Functions, Variables, Methods, Files
```

```
- UPPER_CASE: Constants (environment variables, configuration values)
- kebab-case: Directories, CSS classes

## File Organization
- One component per file
- Related logic in directories with index.ts barrel exports
- Types co-located or in types/ directory
- Test files co-located: `Component.tsx` → `Component.test.tsx`

## Common Patterns
- React Server Components by default; 'use client' only for interactivity
- Server Actions for form mutations
- Route Groups for layout organization
- Loading UI with loading.tsx (Suspense equivalent)
- Error UI with error.tsx (Error Boundary equivalent)

## Testing
- Write tests for all new features
- Follow AAA pattern (Arrange, Act, Assert)
- Use descriptive test names ("should ... when ...")
- Mock network requests with msw

## Documentation
- Add JSDoc comments for public functions and APIs
- Include @example tags for complex usage
- Document breaking changes in PR descriptions
```

---

## 5. .vscode/settings.json

VS Code settings bisa include AI-prompt configurations.

### AI Settings in settings.json

```
{
  // GitHub Copilot
  "github.copilot.enable": {
    "*": true,
    "plaintext": false,
    "markdown": false,
    "yaml": true
  },
  "github.copilot.inlineSuggest.enable": true,
  "github.copilot.chat.localeOverride": "en",
```

```
  // Cursor settings (if using Cursor)
  "cursor.ai.enable": true,
  "cursor.cpp.enableCompletions": true,

  // General AI settings
  "editor.inlineSuggest.enabled": true,
  "editor.suggest.preview": true,
  "editor.suggest.showStatusBar": true,

  // Formatting
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit",
    "source.organizeImports": "explicit"
  },

  // TypeScript
  "typescript.preferences.importModuleSpecifier": "shortest",
  "typescript.preferences.quoteStyle": "single",
  "typescript.suggest.autoImports": true,
  "typescript.updateImportsOnFileMove.enabled": "always",
  "typescript.tsdk": "node_modules/typescript/lib",

  // Files
  "files.exclude": {
    "**/.git": true,
    "**/.next": true,
    "**/node_modules": true,
    "**/dist": true
  },
  "files.associations": {
    "*.css": "tailwindcss"
  }
}
```

---

## 6. Rule Content — Detail Sections

### Coding Style

```
- 2-space indentation
- Single quotes for strings (Prettier)
- No semicolons (default)
- Trailing commas in objects/arrays
```

```
- Arrow functions over `function` keyword
- Template literals over string concatenation Example: `Hello ${name}`
- Destructuring: `const { name, age } = user`
- Spread operator: `const newObj = { ...oldObj, updatedField }`
- Optional chaining: `user?.profile?.name`
- Nullish coalescing: `const val = input ?? defaultValue`
```

**Naming Convention**

```
TypeScript/JavaScript:
- PascalCase: Components, Types, Interfaces, Enums, Classes
- camelCase: Functions, Variables, Methods, Parameters
- UPPER_SNAKE_CASE: Constants, Env vars, Configuration
- Private fields prefix: `_privateField`

Files:
- Components: PascalCase (UserCard.tsx)
- Hooks: camelCase with 'use' prefix (useAuth.ts)
- Utilities: camelCase (formatDate.ts)
- Pages: page.tsx, layout.tsx, loading.tsx, error.tsx
- Tests: *.test.ts, *.spec.ts (co-located)
```

**Test Pattern**

```
Structure:
- Unit tests: Vitest, co-located *.test.ts
- Integration tests: Supertest, co-located *.test.ts
- E2E tests: Playwright, in e2e/ directory

Pattern (AAA):
- Arrange: Setup data, mocks, spies
- Act: Execute function/method
- Assert: Verify expected outcome

Naming:
- describe('ComponentName') for grouping
- it('should x when y') for test cases

Coverage:
- Test success paths
- Test error/edge cases
- Test boundary conditions
- Mock external dependencies
```

**Import Order**

```
Group order (separated by blank line):
1. Node built-ins: fs, path, os
2. External packages: react, express, zod
3. Internal modules: @/components/..., @/lib/...
4. Relative imports: ./utils, ../types
5. Types: import type { X } from '...'

Within each group: alphabetical
No default imports (except for React, express)
Barrel imports via index.ts for directories
```

**Error Handling Pattern**

```
Application errors:
- Custom AppError class with: message, statusCode, code, details
- Business logic returns Result<T, E> (neverthrow)
- Controllers catch errors and pass to errorHandler middleware
- errorHandler middleware sends consistent error response

API errors format:
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Email format is invalid",
   "details": [{ "field": "email", "message": "Invalid format" }]
  }
}

Known vs unknown:
- Known errors: AppError with specific code
- Unknown errors: 500 Internal Server Error + log full error
- Validation errors: 400 with field-level details
- Auth errors: 401 / 403
- Not found: 404
```

---

## 7. Multi-Rule Strategy

Project complex? Break rules into layers.

### Three-Layer Approach

```
project-root/
├── .cursorrules                # Global rules (all files)
├── .cursor/rules/
│   ├── framework.react.mdc        # React-specific rules
│   ├── framework.next.mdc       # Next.js-specific rules
│   ├── database.mdc            # Database-specific rules
│   ├── testing.vitest.mdc         # Testing rules
│   └── project-specific.mdc     # Project-specific rules
└── .cursor/rules/
    └── team-wide/                  # Shared across repos
        └── conventions.mdc
```

### Rule Priority

| Level | Scope | Override |
|---|---|---|
| Global `.cursorrules` | All files | Base rules |
| Framework rules (`*.react.mdc`) | React files | Add framework specifics |
| Project-specific rules | Specific to this project | Override generic rules |
| Team-wide rules | Across repos | Lowest priority |

### Multi-Rule Example

**`.cursor/rules/testing.vitest.mdc`**:

```
<rule>
name: Testing with Vitest
description: Rules for writing tests with Vitest and Testing Library
filters:
  - file: "*.test.ts"
  - file: "*.test.tsx"
  - file: "*.spec.ts"

rules:
  - Use describe/it blocks for test structure
  - Prefer screen queries over render destructuring
  - Use userEvent over fireEvent for user interactions
  - Mock API calls with msw, never mock fetch directly
  - Test accessibility with @testing-library/jest-dom
  - Co-locate test files with source files
</rule>
```

**`.cursor/rules/framework.next.mdc`**:

```
<rule>
name: Next.js App Router Conventions
description: Rules for Next.js 15 App Router patterns
filters:
  - file: "src/app/**"
  - glob: "**/page.tsx"
  - glob: "**/layout.tsx"
  - glob: "**/route.ts"

rules:
  - Pages are server components by default
  - Add 'use client' only for interactive components
  - Use generateMetadata for SEO metadata
  - Route handlers export named functions (GET, POST, etc.)
  - Dynamic routes use [param] folder naming
  - Parallel routes use @slot naming convention
</rule>
```

---

## 8. Latihan: Bikin .cursorrules + copilot-instructions untuk Project TypeScript

### Scenario

Buat rules files untuk project TypeScript Express API (task management).

### Steps

1. **Bikin .cursorrules**: Include coding style, naming, test pattern, import order, error handling — semua spesifik TypeScript
2. **Bikin .github/copilot-instructions.md**: Instructions untuk GitHub Copilot
3. **Bikin rule modular**: Buat at least 2 .cursor/rules/*.mdc files (testing + express)
4. **Bikin .windsurfrules**: Convert cursor rules ke format YAML
5. **Bikin .vscode/settings.json**: AI settings + formatter config

### Deliverable

File di `labs/express-api/`:

- `.cursorrules`
- `.github/copilot-instructions.md`

- `.cursor/rules/testing.mdc`
- `.cursor/rules/express-api.mdc`
- `.windsurfrules`
- `.vscode/settings.json`

---

## Key Takeaways

- `.cursorrules` guide Cursor AI with free-text instructions
- `.windsurfrules` pakai YAML format dengan per-file-pattern rules
- `.github/copilot-instructions.md` instructions untuk GitHub Copilot
- `.vscode/settings.json` bisa configure AI behavior + formatting
- Rule content covers: coding style, naming, testing, imports, error handling
- Multi-rule approach: global → framework → project → team-wide